Oracle Customer to Meter (C2M) Installation & Multi-Environment Build (Dev/Test/Prod)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build 4

1. Solution Overview — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

What this solution is . . . 4

The C2M / OUAF architecture (what you are building) . . . 5

The standard build model (named objects reused everywhere) . . . 5

The certified stack (verify for your version) . . . 6

Benefits . . . 6

Limitations and boundaries . . . 6

What you get . . . 7

2. Requirements Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build 7

2.1 Licensed C2M media and the certification matrix . . . 7

2.2 Certified WebLogic Server 14c installed on the application tier . . . 8

2.3 Certified Java (JDK) on the application tier . . . 8

2.4 Oracle Database available, with schema accounts and Storage.xml . . . 8

2.5 Product OS user, mount points, and file-system layout . . . 9

2.6 Network access and ports between tiers . . . 9

2.7 Per-environment build worksheet (Dev/Test/Prod inputs) . . . 9

2.8 Backup, change control, and a non-production-first sequence . . . 10

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 10

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 11

3. Environment topology (Dev / Test / Prod) & access . . . 11

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 12

6. Collaborating teams & support model . . . 12

3. Installation Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Step 0 — Assess: confirm prerequisites and back up (Assess) . . . 13

Step 1 — Install the OUAF framework (Configure) . . . 13

Step 2 — Install the C2M product application (Configure) . . . 14

Step 3 — Load the C2M database schema (Configure) . . . 14

Step 4 — Create and verify the WebLogic 14c domain (Configure) . . . 15

Step 5 — Configure the environment and validate the build (Validate — pilot gate) . . . 15

Step 6 — Post-install validation and hand-off (Validate) . . . 16

Step 7 — Expand: repeat for Test and Production (Expand) . . . 16

5. Testing Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build 17
Test 1 — Certified stack present (Assess gate) . . . 17
Test 2 — OUAF environment defined correctly . . . 17
Test 3 — Database schema installed and owned by CISADM . . . 18
Test 4 — WebLogic domain healthy . . . 18
Test 5 — C2M application and IWS deployed and active . . . 18
Test 6 — Application-to-database connectivity . . . 18
Test 7 — Build report matches the expected baseline . . . 18
Test 8 — Survives a clean stop/start (reproducibility gate) . . . 19
6. Troubleshooting Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build 20
Symptom: the framework or product installer fails or exits with errors . . . 20
Symptom: `ENVIRON.INI` missing, wrong, or config files look stale . . . 20
Symptom: the database install (`OraDBI`) aborts or logs errors . . . 20
Symptom: `AdminServer` or `c2m_server1` will not reach RUNNING . . . 21
Symptom: the C2M application is deployed but not ACTIVE, or the login page will not load . . . 21
Symptom: application cannot connect to the database (`CISUSER`) . . . 21
Symptom: Test or Prod behaves differently from Dev . . . 21
Symptom: a command was almost run against the wrong environment . . . 22
Symptom: the environment is green on first start but fails after a restart . . . 22
When to escalate to Oracle / My Oracle Support . . . 22
7. Best Practices — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build 22
8. Deliverables Summary — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build 24
Documentation . . . 24
Named objects (reused across the documents and scripts) . . . 25
Automation scripts (WLST / shell / SQL) — five roles . . . 25
What the buyer receives . . . 26
Outcome . . . 26
Appendix A — PowerShell Scripts 27

Executive Summary — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Standing up Oracle Utilities Customer to Meter (C2M) is where utility programs stall. C2M is not a single install — it is a build with three tiers that all have to line up: a **certified platform stack** (OS, Java, WebLogic, database), the **OUAF product installed on a WebLogic 14c domain**, and a **loaded C2M database schema**. Do it by hand once and it works; do it again for the next environment and it drifts — different ports, a hand-edited config, a missed rollup — and now Development, Test, and Production behave differently and every defect is impossible to reproduce.

This solution is the end-to-end, repeatable way to install C2M and build it **consistently across Dev/Test/Prod**. It gives an experienced Oracle/WebLogic administrator the correct ordered procedure — certify the stack, install the OUAF framework, install the C2M product, load the schema with `OraDBI`, create the WebLogic `c2m_domain`, configure and validate — and a set of **named objects reused verbatim in every environment** so the three builds differ only by their host, port, and database inputs. It follows the standard model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**: build and prove non-production first, then build Production under change control with backups and a documented rollback.

The build is **verifiable, not hopeful.** Five automation scripts — WLST/Jython against the WebLogic domain, shell against the OUAF file system, and SQL against the schema — do read-only discovery and validation, produce a build/coverage report per environment, and offer safe, dry-run-by-default remediation. Every stage has an explicit pass/fail gate: certified stack present, ENVIRON.INI correct, schema owned by `CISADM`, `AdminServer` and `c2m_server1` RUNNING, the C2M application ACTIVE, and the whole environment surviving a clean stop and start.

Where a detail is Oracle's — the exact installer prompts, the certification matrix, patch and rollup numbers, gated My Oracle Support notes — this solution points to Oracle's official C2M Installation Guide, C2M Database Administrator's Guide, OUAF Server Administration Guide, and WebLogic 14.1.2 documentation rather than reproducing gated content, and flags every version-specific value (C2M 25.4 / minimum 2.9.x, WebLogic 14c, Java 17, Oracle Database 19c) as "verify for your version." Nothing here is invented.

The result: **Oracle C2M installed and configured consistently in every environment — certified, healthy, and reproducible — proven on a pilot before you scale** — delivered as a self-guided guide plus automation, at a fraction of the cost of a bespoke installation engagement.

1. Solution Overview — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

What this solution is

This is the repeatable, end-to-end way to **install Oracle Utilities Customer to Meter (C2M)** and stand it up consistently across **Development, Test, and Production**. It treats an installation as a build with three tiers that must line up: a **certified platform stack**, the **Oracle Utilities Application Framework (OUAF) product install on WebLogic 14c**, and the **C2M database schema**. It gives you the correct ordered procedure, the exact utilities and console navigation, a set of named objects reused verbatim across every environment, and read-only/dry-run automation so each install is verifiable and reproducible rather than hand-built once and forgotten.

The build is done non-production first (Dev, then Test), proven against pass/fail gates, and only then repeated for Production under change control — the standard model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**.

The C2M / OUAF architecture (what you are building)

C2M is an OUAF application. Every environment you build has the same three layers:

1. **OUAF product home (**`$SPLEBASE`**)** — the installed framework + C2M application code and

configuration on the file system, owned by the product OS user. The environment is defined by `$SPLEBASE/etc/ENVIRON.INI`, which is generated by the installer (never hand-edited) and consumed by every OUAF utility. Each environment has its own `$SPLEBASE`, output directory `$SPLOUTPUT`, and a short environment name `$SPLENVIRON`.

1. **WebLogic 14c domain** — a native WebLogic domain that hosts the C2M application (online web

application, XAI/IWS inbound web services) plus the OUAF batch/threadpool runtime. The domain has an Administration Server and one or more Managed Servers, and is started via WebLogic's own scripts and the OUAF `spl.sh` wrapper.

1. **C2M database schema (Oracle Database)** — the application schema owned by the schema owner account,

with a separate application-runtime user and a read-only user, laid out over the tablespaces defined in the DBA-supplied `Storage.xml`. The schema is installed by the OUAF database installer (`OraDBI`), not by the file-system installer.

> ***Version boundary (verify for your version).*** *This solution is authored against* ***C2M 25.4*** *and, where a step is version-sensitive, against* ***C2M 2.9.x (minimum v2.9)*** *and* ***Oracle WebLogic Server 14c (14.1.2.x)*** *with* ***Java 17*** *and an* ***Oracle Database 19c*** *client, per Oracle's official C2M Installation Guide, C2M Database Administrator's Guide, OUAF Server Administration Guide, and WebLogic 14.1.2 documentation.* ***Confirm the exact certified combination for your release against Oracle's certification matrix on My Oracle Support (MOS) before you build.*** *MOS-gated patches, rollups, and notes are pointed to, never reproduced here.*

The standard build model (named objects reused everywhere)

This solution names every object it creates and **reuses those exact names** in the Installation, Testing, and Troubleshooting guides and in the scripts. Adopt these names, or substitute your own convention once and apply it consistently. Placeholders are written without angle brackets (for example `your-host`, `your-db`, `your-mount`), never invented values.

Object	Name used in this solution	Notes
Environments (`$SPLENVIRON`)	`C2MDEV`, `C2MTST`, `C2MPRD`	Development, Test, Production
Product OS user	`cissys`	Owns `$SPLEBASE`; runs all OUAF utilities
Product home base	`/opt/oracle/spl`	`$SPLEBASE` = `/opt/oracle/spl/C2MDEV`, etc.
Output base	`/opt/oracle/splout`	`$SPLOUTPUT` per environment

Object	Name used in this solution	Notes
WebLogic domain (per env)	`c2m_domain`	Native WebLogic 14c domain under `$SPLEBASE`
WebLogic Admin Server	`AdminServer`	Domain Administration Server
WebLogic Managed Server	`c2m_server1`	Hosts the C2M online + IWS applications
DB schema owner	`CISADM`	Owns the C2M application schema
DB application user	`CISUSER`	Runtime application connection user
DB read-only user	`CISREAD`	Read-only privileges to the schema
Application security role	`cisusers`	WebLogic role granted online access

The database, WebLogic ports, and mount points differ per environment; the object **names** stay the same so scripts and runbooks are environment-agnostic and you target an environment by its `$SPLENVIRON`.

The certified stack (verify for your version)

Before any install, the host OS, JDK, Oracle Database, WebLogic, and C2M release must be a combination **certified together** for your C2M release. As a representative baseline for C2M 25.4 / 2.9.x — **which you must confirm on MOS** — that is: a certified Linux/UNIX (or Windows) host, **Java 17**, **Oracle WebLogic Server 14c (14.1.2.x)** installed from the FMW/WLS distribution, and an **Oracle Database 19c** (with the 19c client on the application tier). Uncertified stacks cause install and patch failures and lose supportability, so the certified matrix is the first gate, not an afterthought.

Benefits

• **Consistency across Dev/Test/Prod** — the same ordered procedure and the same named objects in every

environment, so promotions and defect reproduction are not confounded by build drift.

• **Verifiable installs** — discovery, validation, and reporting scripts confirm the product home, the

WebLogic domain, and the schema are healthy against explicit pass/fail gates.

• **Production-safe** — non-production first, verified backups before disruptive steps, and a documented

rollback; remediation is dry-run by default.

• **Reproducible** — because ENVIRON.INI, the domain, and the schema are built the documented way, an

environment can be rebuilt from the same inputs rather than reverse-engineered.

Limitations and boundaries

• **Not a substitute for Oracle's licensed media, MOS access, or the official guides.** Exact installer

prompts, the certification matrix, patch numbers, and gated notes are Oracle's — this solution points to them and gives the correct procedure and object model around them.

- **Version-sensitive.** Release numbers, menu-item numbers, and default ports change between releases;

every such specific here is flagged "verify for your version."

- **Assumes prerequisites exist.** Provisioned hosts, the WebLogic 14c install, an available Oracle

Database, the DBA-supplied `Storage.xml`, and network access between tiers are prerequisites (see the Requirements Guide), not steps built by this solution.

- **Configuration beyond initial build** (SSO/SSL, Coherence/JMS/IWS tuning, batch scheduling,

monitoring) is deferred to the dedicated solutions in this Oracle catalog; this one delivers a correct, healthy baseline install per environment.

What you get

An installation and multi-environment build guide (this document set as Word + PDF), five automation scripts (WLST/Jython, shell, and SQL) for discovery, validation, reporting, safe remediation, and environment targeting, and a repeatable process that produces a certified, healthy, reproducible C2M build in Dev, Test, and Production.

2. Requirements Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Verify every requirement before you start the Installation Guide. Each entry follows: **Description**, **Why**, **Verify**, and **Implement if missing**. Category-wide prerequisites (entitlement, certified stack, environment topology, tooling, change control, collaborating teams) are in `global-requirements-oracle.md`; the requirements below are the ones specific to building a C2M installation across Dev/Test/Prod. Placeholders are written without angle brackets (for example `your-host`, `your-mount`, `your-db`).

> *Version note: specifics such as C2M 25.4 / minimum 2.9.x, WebLogic 14c (14.1.2.x), Java 17, and Oracle Database 19c are **flagged "verify for your version"** — always confirm against Oracle's certification matrix on My Oracle Support (MOS) for your exact release.*

2.1 Licensed C2M media and the certification matrix

- **Description:** The licensed C2M installation media (framework, product, and database components) for

your release, and the matching **certification/compatibility matrix** that pins certified OS, Java, WebLogic, and Oracle Database versions together.

- **Why:** C2M/OUAF only supports certified combinations; the media and matrix are the authoritative

inputs to the whole build.

- **Verify:** Confirm you can download the C2M framework, product, and database media for your release

from Oracle Software Delivery / MOS, and locate the certification matrix for that release.

- **Implement if missing:** Obtain the entitlement and MOS access; do not proceed on uncertified media.

2.2 Certified WebLogic Server 14c installed on the application tier

- **Description: Oracle WebLogic Server 14c (14.1.2.x — verify for your version)** installed into an Oracle Home on each application-tier host, from the WLS/FMW distribution, before the C2M product install. C2M creates a **native** WebLogic domain into this install.
- **Why:** The C2M online application, inbound web services, and batch runtime are hosted in a WebLogic domain; the install expects a certified WebLogic Home present.
- **Verify:** Confirm the WebLogic Home exists and its version matches the matrix (for example inspect the registry/inventory or run the WLS version check). Confirm the OS product user can read it.
- **Implement if missing:** Install certified WebLogic 14c per Oracle's WebLogic 14.1.2 installation documentation before the C2M product step.

2.3 Certified Java (JDK) on the application tier

- **Description:** A certified **JDK — Java 17 for C2M 25.4 / 2.9.x (verify for your version)** installed on each application-tier host, with `JAVA_HOME` and `PATH` set for the product OS user.
- **Why:** OUAF utilities, WebLogic, and the C2M application all run on the certified JDK; a mismatched Java version breaks install and runtime.
- **Verify:** As the product OS user, run `java -version` and confirm it matches the certified JDK.
- **Implement if missing:** Install the certified JDK and set `JAVA_HOME`/`PATH` before installing.
- **Note:** The **database installer (`OraDBI`)** may require a specific Java per your release (some releases require an older JDK to run the DB install tool) — verify the DB install step's Java requirement in the C2M Database Administrator's Guide separately from the application-tier Java.

2.4 Oracle Database available, with schema accounts and Storage.xml

- **Description:** A reachable **Oracle Database (19c — verify for your version)** for each environment, plus the plan for the three C2M schema accounts — schema owner `CISADM`, application user `CISUSER`, read-only user `CISREAD` — and the DBA-supplied `Storage.xml` defining the tablespaces the C2M schema will use.
- **Why:** The C2M database installer (`OraDBI`) creates/loads the application schema into these accounts over the tablespaces named in `Storage.xml`; if a named tablespace is missing the install aborts.
- **Verify:** Confirm database connectivity (TNS/`tnsping`, SQL*Plus login), that a DBA will create the tablespaces named in your `Storage.xml`, and that you have (or can create) `CISADM`/`CISUSER`/`CISREAD`.

• **Implement if missing:** Have the DBA provision the database, tablespaces, and accounts, and hand you a

populated `Storage.xml` before the database install step.

2.5 Product OS user, mount points, and file-system layout

• **Description:** A dedicated **product OS user (**`cissys`**)** on each application-tier host, and the

planned **mount points** for the environment home (`$SPLEBASE`, under `/opt/oracle/spl/your-env`), the output directory (`$SPLOUTPUT`, under `/opt/oracle/splout/your-env`), and the **log mount point** — the log directory must already exist before the framework install.

• **Why:** OUAF is installed and run as the product user under $SPLEBASE; the installer requires the

mount points (and an existing log directory) to lay down and configure the environment.

• **Verify:** Confirm the `cissys` account exists on each host, owns the mount points, and that the log

mount point directory already exists.

• **Implement if missing:** Create the OS user and mount points and pre-create the log directory before

running the installer.

2.6 Network access and ports between tiers

• **Description:** Network reachability between the application tier and the database (Oracle listener

port), plus the planned **WebLogic ports** (Administration Server, Managed Server `c2m_server1`, and any SSL/IWS ports) open per environment and not already in use on the host.

• **Why:** The domain cannot start and the application cannot reach its schema without these paths; port

collisions on a shared host block domain startup.

• **Verify:** From each app host confirm you can reach the DB listener; confirm the chosen WebLogic ports

are free on the host (they are not already bound by another domain or service).

• **Implement if missing:** Open the firewall paths and choose free ports before creating the domain.

2.7 Per-environment build worksheet (Dev/Test/Prod inputs)

• **Description:** A completed **install worksheet per environment** capturing, for `C2MDEV`, `C2MTST`,

and `C2MPRD`, the exact values the installer prompts for: environment name (`$SPLENVIRON`), mount points, WebLogic host and ports, WebLogic admin credentials, database name/host/service and schema user, and the keystore options.

• **Why:** The OUAF installer is prompt-driven; having every value decided up front is what makes the

three builds consistent rather than improvised, and lets you script validation against known-expected values.

• **Verify:** Confirm a filled worksheet exists for each environment and that Test and Prod differ from

Dev only in host/port/DB values, not in structure or object names.

• **Implement if missing:** Complete the worksheet from Oracle's installation-worksheet section of the C2M Installation Guide before running any install.

2.8 Backup, change control, and a non-production-first sequence

• **Description:** Verified **backups** (any existing domain config, OUAF product/config, and database) before disruptive steps, a **change window/approval** for the Production build, and agreement to build and prove **Dev then Test first**, Production last.

• **Why:** Installs and schema loads are high-impact; the solution follows Assess → Configure → Pilot → Validate → Expand with a documented rollback, and Production is only built after non-prod passes its gates.

• **Verify:** Confirm a backup/restore path and an approved change window for the Production install.

• **Implement if missing:** Establish backups and change control, and sequence Dev/Test before Prod, before touching Production.

Requirements checklist

```
Licensed media + certification matrix -> WebLogic 14c installed -> Certified
Java -> Database + CISADM/CISUSER/CISREAD + Storage.xml -> Product OS user +
mount points (log dir exists) -> Network + free ports -> Per-env worksheet ->
Backup + change control + non-prod first
```

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Installation Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Follow these steps in order, **per environment**, building `C2MDEV` first, then `C2MTST`, then `C2MPRD` under change control. Work as the product OS user `cissys` on the application tier and with the DBA for the database step. The supplied scripts assist at the read-only and validation steps; the remediation script is dry-run by default.

Exact installer prompts, menu-item numbers, default ports, patch/rollup numbers, and the certification matrix are **Oracle's and version-specific** — this guide gives the correct ordered procedure and the named objects, and points to Oracle's C2M Installation Guide, C2M Database Administrator's Guide, OUAF Server Administration Guide, and the WebLogic 14.1.2 documentation for the exact screens. Placeholders are written without angle brackets (for example `your-host`, `your-db`, `your-mount`). Every version specific below is flagged **"verify for your version."**

> *Model: certify the stack, install the framework, install the product, create/verify the WebLogic domain, load the database schema, configure the environment, validate — proven on `C2MDEV` and `C2MTST` before `C2MPRD`. Standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.***

Named objects produced/used by this guide (reuse these exact names):

• Environments: `C2MDEV`, `C2MTST`, `C2MPRD` (`$SPLENVIRON`)

- Product home: $SPLEBASE = /opt/oracle/spl/your-env; output: $SPLOUTPUT = /opt/oracle/splout/your-env
- WebLogic domain c2m_domain, Admin Server AdminServer, Managed Server c2m_server1
- DB accounts CISADM (owner), CISUSER (app), CISREAD (read-only); security role cisusers

Step 0 — Assess: confirm prerequisites and back up (Assess)

1. Confirm every item in the Requirements Guide for the environment you are about to build: certified media + matrix, WebLogic 14c installed, certified Java, database + accounts + Storage.xml, product OS user + mount points (log directory already exists), free WebLogic ports, and a completed worksheet.

1. Run OC2MINST-Discovery.sh on the target host to capture the starting state read-only: OS user, JAVA_HOME/java -version, WebLogic Home and version, mount points and free space, listener reachability to your-db, and whether the chosen WebLogic ports are free. Save the output as the Build Baseline – your-env – Assess record.

1. For any rebuild or for Production, take verified **backups** first (existing $SPLEBASE, any existing domain config, and the database) and confirm the restore path. This is the rollback anchor.

1. Set the shell for the install: as cissys, ensure JAVA_HOME and PATH point at the certified JDK (Java 17 — verify for your version).

Gate: all prerequisites present, baseline captured, backups verified. Do not proceed otherwise.

Step 1 — Install the OUAF framework (Configure)

1. Extract the framework media to a temporary directory and change into it (for example your-temp/FW-Vxx).

1. Run the installer as cissys: ksh ./install.sh (UNIX) or install.cmd (Windows). The installer is an interactive, menu-driven utility; drive it from your completed worksheet. Verify for your version.

1. Complete the framework menu blocks in order (numbers vary by release — **verify for your version**):

- **Environment ID, Roles, Third-Party Software** — environment name ($SPLENVIRON, e.g. C2MDEV), Java Home, WebLogic Home.
- **Keystore Options** — the OUAF keystore (store type JCEKS, alias ouaf.system, HMAC alias ouaf.system.hmac; accept documented defaults unless you have a keystore standard).
- **Environment Installation Options** — the environment **mount point** ($SPLEBASE under /opt/oracle/spl/C2MDEV), the **log mount point** (must already exist), output directory ($SPLOUTPUT), and web application server type **WebLogic**.
- **Environment parameters** — web server host/ports, database name and schema user; these are written to $SPLEBASE/etc/ENVIRON.INI. Press **P** to proceed.

1. The installer automatically runs `initialSetup.sh` (in `$SPLEBASE/bin`), which builds the

configuration files, keystore, and WAR/EAR from the ENVIRON.INI values, then runs `splenviron.sh -e your-env` to set the environment variables.

1. **Do not hand-edit** `ENVIRON.INI`**.** To change any value later, re-run `configureEnv.sh` then

`initialSetup.sh` so the templates are regenerated consistently.

Gate: framework install completes with no errors; `$SPLEBASE/etc/ENVIRON.INI` exists and names the correct environment, mount points, and database.

Step 2 — Install the C2M product application (Configure)

1. In a new session, initialize the environment: `$SPLEBASE/bin/splenviron.sh -e your-env` (e.g.

`C2MDEV`). Stop the environment if it is running.

1. Extract the C2M product media (for example `jar -xvf C2M-Vxx-MultiPlatform.jar`) and change into the

product install directory.

1. Run `ksh ./install.sh` (UNIX) / `install.cmd` (Windows). On the configuration menu, if no changes are

needed, choose **P** to proceed. Verify for your version.

1. Re-apply configuration and rebuild templates:

```
splenviron.sh -e your-env
configureEnv.sh            # review values; type P and Enter if no changes
splenviron.sh -e your-env
initialSetup.sh            # rebuilds config/EAR from ENVIRON.INI
```

1. Review and, per Oracle's guidance, incorporate `$SPLEBASE/tools/examples/bin/setUserOverride.sh` into

the WebLogic domain start-up (Step 4). This carries OUAF-required JVM/classpath settings into the domain.

1. Apply any **post-release rollup/patches** required for your release — extract the rollup and run its

installer (for example `installSFgroup.sh`). **Patch numbers are MOS-gated; obtain them from MOS and follow the rollup's own readme** — do not skip required rollups.

Gate: product install completes with no errors; `initialSetup.sh` rebuilt the EAR/config cleanly.

Step 3 — Load the C2M database schema (Configure)

Perform with the DBA. The database schema is installed by the OUAF database installer, **not** by the file-system installer above.

1. Confirm the DBA has created the tablespaces named in your `Storage.xml` and the target database is

reachable. If a named tablespace is missing, the install aborts by design.

1. From the C2M **database** media, use the `Install-Upgrade` folder. Set `JAVA_HOME`/`PATH`/`CLASSPATH`

as the DB install step requires (**the DB installer's Java may differ from the app-tier Java — verify for your version**).

1. Run the OUAF database installer `OraDBI` for an initial install. It prompts for the schema owner

(`CISADM`), the application user (`CISUSER`), and the read-only user (`CISREAD`) and their passwords, validates the `Storage.xml` tablespaces, and loads the schema and seed data. Verify for your version.

1. Check the logs in `Install-Upgrade/logs` for errors; the schema load is not complete until the logs

are clean.

Gate: `OraDBI` completes; logs are error-free; `CISADM`/`CISUSER`/`CISREAD` exist and the schema objects are present (see Testing).

Step 4 — Create and verify the WebLogic 14c domain (Configure)

C2M uses a **native** WebLogic 14c domain named `c2m_domain`.

1. Create the domain into the WebLogic 14c Home using Oracle's WebLogic Configuration Wizard (or the

documented WLST offline flow), per the C2M Installation Guide and WebLogic 14.1.2 documentation. Name the domain `c2m_domain`, the Administration Server `AdminServer`, and the Managed Server `c2m_server1`, using the ports from your worksheet. Verify for your version.

1. Incorporate the OUAF `setUserOverride.sh` settings (Step 2.5) into the domain's start-up so the C2M

application deploys with the correct JVM/classpath. Confirm memory arguments are sized for C2M per Oracle's guidance.

1. Start the domain: from `$DOMAIN_HOME/bin` start the Admin Server (`./startWebLogic.sh`) and then

`c2m_server1`; or use the OUAF wrapper `spl.sh -start` to start the environment. Verify for your version.

1. Deploy the C2M application and inbound web services as documented (the EAR built by

`initialSetup.sh`; `iwsdeploy.sh` for inbound web services where applicable).

1. In the WebLogic Administration Console (`http://your-host:your-admin-port/console`), grant online

application access to the `cisusers` security role on first start-up, per Oracle's guidance.

Gate: `AdminServer` and `c2m_server1` reach RUNNING; the C2M application and IWS are deployed and in an active state.

Step 5 — Configure the environment and validate the build (Validate — pilot gate)

1. Confirm application-to-database connectivity: the C2M application connects to the schema as `CISUSER`

and the online login page is reachable.

1. Run `OC2MINST-Validation.py` (WLST, against the domain) to confirm the build against the expected

named objects: `AdminServer` and `c2m_server1` RUNNING and the C2M application/IWS ACTIVE. Save the result as `Build Validation – your-env – Validate`. (Use `OC2MINST-ScopeHelper.sh` first to confirm `ENVIRON.INI` is present/correct and `$SPLEBASE`/`$SPLOUTPUT` are populated for the environment.)

1. Run the C2M schema check `OC2MINST-Report.sql` (as a read-only user against the schema) to confirm

`CISADM` owns the expected objects, `CISUSER`/`CISREAD` exist, and the installed version/release matches the intended level — this is the **build/schema report** for the environment.

1. If any check fails, run `OC2MINST-Remediation.sh` in its default **dry-run** mode to see the planned

safe fixes (for example: correct file ownership/permissions on `$SPLEBASE`, restart a server that is ADMIN/STANDBY rather than RUNNING, re-run `initialSetup.sh -t` to regenerate templates). Apply only after review with `-apply`.

Pilot gate (`C2MDEV`, then `C2MTST`): all Validation and Report checks pass. See the Testing Guide for the full pass/fail criteria. Do not build Production until Dev and Test both pass.

Step 6 — Post-install validation and hand-off (Validate)

1. Record the environment in your build register with its `$SPLENVIRON`, hosts/ports, DB service, and the

validated report — this is the reproducible definition of the environment.

1. Confirm the environment can be cleanly stopped and started (`spl.sh -stop` / `spl.sh -start`) and that

validation still passes after a restart — proving the build survives a bounce, not just first start.

1. Hand the environment to the teams that own the next layers (SSO/SSL, Coherence/JMS/IWS tuning, batch

scheduling, monitoring) — those are the dedicated solutions in this catalog. This solution's scope is a certified, healthy baseline install.

Step 7 — Expand: repeat for Test and Production (Expand)

1. After `C2MDEV` passes, repeat Steps 0–6 for `C2MTST`, then — inside an approved change window with

backups verified — for `C2MPRD`. Reuse the **same object names**; change only host/port/DB values from each environment's worksheet.

1. Use `OC2MINST-ScopeHelper.sh` to target the correct environment for every run (it resolves the

$SPLENVIRON, $SPLEBASE, domain, and DB for the named environment) so a command intended for C2MTST never runs against C2MPRD.

1. Compare each environment's OC2MINST-Report.sql output and Discovery baseline to Dev's to confirm the

builds match structurally — the point of the multi-environment build is that Dev, Test, and Prod differ only by their host/port/DB inputs.

Order of operations

```
Assess + backup -> Framework install (install.sh -> initialSetup.sh) ->
Product install (configureEnv -> initialSetup) -> DB schema (OraDBI:
CISADM/CISUSER/CISREAD over Storage.xml) -> WebLogic domain c2m_domain
(AdminServer + c2m_server1, deploy C2M) -> Configure + validate (pilot gate)
-> Post-install validation -> Expand to Test then Prod
```

5. Testing Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Run these tests after each environment is built (Installation Guide Steps 1–5) and again after any restart or change, **per environment** — C2MDEV and C2MTST must both pass before C2MPRD is built. Each test states what it proves, how to run it, and an explicit **pass / fail** gate. The read-only scripts (OC2MINST-Discovery.sh, OC2MINST-Validation.py, OC2MINST-Report.sql) do the checking; nothing here changes the environment.

Use OC2MINST-ScopeHelper.sh first to confirm you are targeting the intended environment ($SPLENVIRON, $SPLEBASE, c2m_domain, and DB service) so a test never runs against the wrong tier.

Test 1 — Certified stack present (Assess gate)

- **Proves:** the host runs a certified OS/Java/WebLogic/Database combination for the release.
- **Run:** OC2MINST-Discovery.sh and compare java -version, the WebLogic Home version, and the DB

version to the certification matrix for your release.

- **Pass:** all four match a certified combination. **Fail:** any component off the matrix — stop and

align the stack before continuing (an uncertified stack is unsupported).

Test 2 — OUAF environment defined correctly

- **Proves:** the framework/product install produced a valid environment.
- **Run:** confirm $SPLEBASE/etc/ENVIRON.INI exists and names the correct $SPLENVIRON

(C2MDEV/C2MTST/C2MPRD), mount points, and database; confirm $SPLEBASE and $SPLOUTPUT are populated. OC2MINST-ScopeHelper.sh resolves and reports these for the environment.

- **Pass:** ENVIRON.INI present and correct, directories populated, no build errors in the install logs.

Fail: missing/incorrect ENVIRON.INI or empty $SPLEBASE — re-run `configureEnv.sh` + `initialSetup.sh` per the Installation Guide.

Test 3 — Database schema installed and owned by CISADM

- **Proves:** the schema loaded over the `Storage.xml` tablespaces and the accounts exist.
- **Run:** `OC2MINST-Report.sql` as a read-only user against the schema; confirm `CISADM` owns the expected objects, `CISUSER` and `CISREAD` exist, and the installed release/version row matches the intended level. Confirm the `OraDBI` logs in `Install-Upgrade/logs` were clean.
- **Pass:** accounts present, `CISADM` owns the C2M objects, version matches, logs clean. **Fail:** missing objects/accounts or version mismatch — review the `OraDBI` logs and re-run the DB install step with the DBA.

Test 4 — WebLogic domain healthy

- **Proves:** the `c2m_domain` domain and its servers are up.
- **Run:** `OC2MINST-Validation.py` via WLST against the domain; confirm `AdminServer` and `c2m_server1` are **RUNNING** (not ADMIN/STANDBY/FAILED). Cross-check in the Administration Console.
- **Pass:** both servers RUNNING with no health warnings. **Fail:** a server not RUNNING — check the server logs and the incorporated `setUserOverride.sh`/memory settings; `OC2MINST-Remediation.sh` (dry-run) will propose a safe restart.

Test 5 — C2M application and IWS deployed and active

- **Proves:** the application is deployed and serving.
- **Run:** confirm via WLST/console that the C2M application and inbound web services are **ACTIVE** on `c2m_server1`; load the online login page at `http://your-host:your-port/` and confirm it renders.
- **Pass:** application ACTIVE and login page reachable. **Fail:** application not ACTIVE or page unreachable — check deployment state, the `cisusers` role grant, and app-to-DB connectivity.

Test 6 — Application-to-database connectivity

- **Proves:** the running application reaches its schema as `CISUSER`.
- **Run:** authenticate to the online login page; confirm no data-source/connection errors in the managed-server log; confirm the WebLogic data source targeting `c2m_server1` is healthy.
- **Pass:** login succeeds and the data source is healthy. **Fail:** connection errors — verify the DB service, listener reachability, and `CISUSER` credentials.

Test 7 — Build report matches the expected baseline

• **Proves:** the environment matches its intended definition and (for Test/Prod) matches Dev structurally.

• **Run:** combine the `OC2MINST-Discovery.sh` baseline (stack versions, ports), `OC2MINST-Validation.py` (domain + application state), and `OC2MINST-Report.sql` (schema state) into the build report; compare to the `Build Baseline – your-env – Assess` and to Dev's report.

• **Pass:** report matches the expected baseline; Test/Prod differ from Dev only in host/port/DB values. **Fail:** structural drift — reconcile before promoting workloads.

Test 8 — Survives a clean stop/start (reproducibility gate)

• **Proves:** the build is stable, not just green on first start.

• **Run:** `spl.sh -stop` then `spl.sh -start` for the environment; re-run Tests 4–7.

• **Pass:** the environment comes back and all checks pass after the bounce. **Fail:** it does not come back cleanly — resolve before declaring the environment done or building the next tier.

Test summary

#	Test	Pass gate
1	Certified stack	OS/Java/WebLogic/DB all on the matrix
2	OUAF environment	ENVIRON.INI correct, $SPLEBASE populated
3	DB schema	`CISADM` owns objects; `CISUSER`/`CISREAD` exist; version matches; clean logs
4	WebLogic domain	`AdminServer` + `c2m_server1` RUNNING
5	Application + IWS	C2M ACTIVE; login page renders
6	App-to-DB	login works; data source healthy
7	Build report	matches baseline; Test/Prod ≈ Dev structurally
8	Stop/start	environment returns; Tests 4–7 still pass

Overall pass: all eight pass for `C2MDEV` and `C2MTST` before `C2MPRD` is built, and all eight pass for `C2MPRD` inside its change window.

6. Troubleshooting Guide — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Use `OC2MINST-Discovery.sh` for host/prereq state, `OC2MINST-ScopeHelper.sh` to confirm you are on the intended environment, `OC2MINST-Validation.py` (WLST) for the domain and application, and `OC2MINST-Report.sql` for the schema. Apply fixes with `OC2MINST-Remediation.sh` — **dry-run by default**, `-apply` only after review. Oracle-specific internals (exact installer prompts, the certification matrix, patch/rollup numbers, gated My Oracle Support notes) are Oracle's; this guide gives the correct diagnostic path and points to Oracle's C2M Installation Guide, C2M Database Administrator's Guide, OUAF Server Administration Guide, and WebLogic 14.1.2 documentation for the details. Every version-specific value is **"verify for your version."**

Work the stages in build order — stack, framework, product, database, domain, application — because a failure at a later stage is usually caused by an unmet gate at an earlier one.

Symptom: the framework or product installer fails or exits with errors

- **Likely causes:** uncertified Java/WebLogic; the log mount point does not exist; wrong mount-point ownership; `JAVA_HOME`/`PATH` not set for `cissys`; a worksheet value entered wrong.
- **Do:** run `OC2MINST-Discovery.sh -e your-env` and fix any CHECK first (Test 1/Test 2 in Testing).

Confirm the log directory exists before installing. Confirm `java -version` matches the certified JDK. Re-run the installer from the completed worksheet.

- **Do not:** hand-edit `ENVIRON.INI` to "fix" a value — re-run `configureEnv.sh` then `initialSetup.sh`

so the templates regenerate consistently (`OC2MINST-Remediation.sh -t templates`).

Symptom: `ENVIRON.INI` missing, wrong, or config files look stale

- **Likely causes:** `initialSetup.sh` did not complete; a value was hand-edited; templates not rebuilt after a change.
- **Do:** confirm `$SPLEBASE/etc/ENVIRON.INI` exists and names the correct environment/DB (`OC2MINST-ScopeHelper.sh -e your-env`). Regenerate templates with `OC2MINST-Remediation.sh -e your-env -t templates` (dry-run, then `-apply`), which runs `initialSetup.sh -t` under the environment.
- **Escalate:** if ENVIRON.INI is structurally wrong, re-run `configureEnv.sh` to set the values, then `initialSetup.sh`, per the Installation Guide.

Symptom: the database install (`OraDBI`) aborts or logs errors

- **Likely causes:** a tablespace named in `Storage.xml` does not exist; wrong DB Java (the DB installer may need a different JDK than the app tier); insufficient privileges; connectivity to `your-db` down.

• **Do:** with the DBA, confirm every tablespace in `Storage.xml` exists (a missing one aborts by design). Confirm the DB install step's Java requirement (**verify for your version** — it may differ from the app-tier Java). Read the logs in `Install-Upgrade/logs` — the schema load is not complete until they are clean. Re-run the DB install step.

• **Verify after:** `OC2MINST-Report.sql` — `CISADM` exists and owns objects, `CISUSER`/`CISREAD` exist, invalid-object count is 0 (or explained), installed version matches (Testing Test 3).

Symptom: `AdminServer` or `c2m_server1` will not reach RUNNING

• **Likely causes:** a WebLogic port already in use on the host; `setUserOverride.sh` settings not incorporated into the domain start-up; memory arguments too small for C2M; a bad admin credential.

• **Do:** re-run `OC2MINST-Discovery.sh -e your-env -l your-ports` to confirm the ports are free.

Confirm `$SPLEBASE/tools/examples/bin/setUserOverride.sh` is incorporated into the domain start-up and that memory is sized per Oracle's guidance. Read the server logs. Offer a controlled restart with `OC2MINST-Remediation.sh -e your-env -t restart` (dry-run, then `-apply`), then re-check with `OC2MINST-Validation.py`.

• **Escalate:** persistent FAILED/ADMIN state → follow Oracle's WebLogic 14.1.2 server-troubleshooting documentation with the server log's specific error.

Symptom: the C2M application is deployed but not ACTIVE, or the login page will not load

• **Likely causes:** the EAR was not rebuilt after a change; the `cisusers` role was not granted online access on first start-up; the application-to-database data source is unhealthy.

• **Do:** confirm the application deployment state via `OC2MINST-Validation.py` (Testing Test 5). In the Administration Console, confirm the `cisusers` role grant. If a config change preceded this, rebuild the EAR (`initialSetup.sh`) and redeploy. Check app-to-DB connectivity (next symptom).

Symptom: application cannot connect to the database (`CISUSER`)

• **Likely causes:** DB service/listener down or unreachable from the app tier; wrong `CISUSER` credentials; the data source not targeted to `c2m_server1`.

• **Do:** from the app host confirm listener reachability (`OC2MINST-Discovery.sh -d your-db-host`).

Confirm the WebLogic data source is healthy and targeted to `c2m_server1`. Verify `CISUSER` credentials. Testing Test 6 is the pass/fail gate.

Symptom: Test or Prod behaves differently from Dev

• **Likely causes:** build drift — a different port scheme, a hand-edited config, a rollup applied to one tier but not another, or object names that were not reused.

• **Do:** compare the `OC2MINST-Discovery.sh` baselines and `OC2MINST-Report.sql` output across the environments (Testing Test 7). Confirm the object names are identical and only host/port/DB differ. Confirm the same required rollups are applied in every tier. Reconcile before promoting workloads.

Symptom: a command was almost run against the wrong environment

• **Likely causes:** multiple environments on shared infrastructure; ambiguous shell state.

• **Do:** always run `OC2MINST-ScopeHelper.sh -e your-env` first; for Production it refuses an OK-to-proceed unless you pass `-confirm C2MPRD`. Make this the habitual first step of any change.

Symptom: the environment is green on first start but fails after a restart

• **Likely causes:** a manual runtime change that was never persisted to the built configuration; a start-up ordering or dependency problem.

• **Do:** run Testing Test 8 — `spl.sh -stop` then `-start` — and re-validate (Tests 4–7). Fold any manual fix back into the documented build (via `configureEnv.sh`/`initialSetup.sh`) so it survives a bounce. A build that does not come back cleanly is not done.

When to escalate to Oracle / My Oracle Support

• Install/patch failures tied to the certification matrix, a required rollup, or a gated note.

• `OraDBI`/schema errors that persist after tablespace and Java prerequisites are confirmed.

• WebLogic server failures whose logs point to a product defect rather than configuration.

Gather the relevant logs (`Install-Upgrade/logs`, the WebLogic server logs, and the installer output) and the exact release/version before opening the service request.

Triage order

```
ScopeHelper (right env?) -> Discovery (stack/ports/prereqs) ->
framework/product install logs -> OraDBI logs + Report.sql (schema) ->
Validation.py (domain + app) -> app-to-DB -> Remediation (dry-run -> apply) ->
compare tiers -> restart test -> escalate to Oracle/MOS
```

7. Best Practices — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

• **Certify the stack before you touch an installer.** Confirm OS, Java, WebLogic 14c, and Oracle

Database are a combination certified *together* for your C2M release against the MOS certification matrix. This is the first gate, not a post-install check — an uncertified stack fails installs, blocks patches, and loses supportability.

- **Build non-production first, always.** Build and fully validate `C2MDEV`, then `C2MTST`, and only then `C2MPRD` inside an approved change window. Production should be the *third* time you run the procedure, not the first.
- **Decide every value up front on a per-environment worksheet.** The OUAF installer is prompt-driven; improvising values is how Dev, Test, and Prod drift apart. Fill the worksheet from Oracle's installation-worksheet section so the three builds differ only by host/port/DB.
- **Reuse the same object names in every environment.** Keep `C2MDEV`/`C2MTST`/`C2MPRD`, `cissys`, `c2m_domain`, `AdminServer`, `c2m_server1`, `CISADM`/`CISUSER`/`CISREAD`, and the `cisusers` role identical across tiers. Environment-agnostic names make scripts, runbooks, and defect reproduction work across all three.
- **Never hand-edit `ENVIRON.INI`.** It is generated. To change a value, re-run `configureEnv.sh` then `initialSetup.sh` so every templated config file is regenerated consistently. Editing the file by hand drifts the templates and creates subtle, hard-to-find breakage.
- **Keep the database install separate and DBA-owned.** The schema is loaded by `OraDBI` over the tablespaces named in the DBA's `Storage.xml`, with its own (possibly different) Java. Confirm the tablespaces exist first — a missing one aborts the install — and always check the `Install-Upgrade/logs` before calling the schema done.
- **Incorporate `setUserOverride.sh` into the domain start-up.** The OUAF-required JVM/classpath settings from `$SPLEBASE/tools/examples/bin/setUserOverride.sh` must reach the WebLogic domain, or the C2M application will not deploy or run correctly. Verify it is applied after every product install or rollup.
- **Choose free ports and check them before creating the domain.** On shared hosts, a port collision silently blocks domain startup. Verify the Admin/Managed/SSL ports are free (the discovery script checks) before you create `c2m_domain`.
- **Back up before every disruptive step and know your rollback.** Before an install, rollup, or schema load — especially in Production — back up $SPLEBASE, the domain config, and the database, and confirm the restore path. The backup is the rollback anchor for Assess → Configure → Pilot → Validate → Expand.
- **Apply required rollups; get patch numbers from MOS.** Post-release rollups are often mandatory for a supported install. They are MOS-gated — pull the numbers and readmes from MOS and follow the rollup's own instructions; never skip a required rollup to save time.
- **Prove the build survives a bounce.** A green first-start is not a finished environment. Stop and start (`spl.sh -stop`/`-start`) and re-validate; a build that does not come back cleanly is not done.

• **Keep secrets out of scripts and configs.** Read database and WebLogic credentials from prompts, the

OUAF keystore, or the environment — never hard-code them into scripts, ENVIRON.INI overrides, or the worksheet you circulate. The supplied automation reads credentials at run time and stores none.

• **Remediate dry-run first.** Run `OC2MINST-Remediation.sh` with no flag to see the planned safe fixes,

review them, then re-run with `-apply`. Never let an unattended fix touch a Production environment.

• **Record each environment as a reproducible definition.** Save the validated build report with the

`$SPLENVIRON`, hosts/ports, DB service, and versions. The goal of a multi-environment build is that any tier can be rebuilt from its recorded inputs, not reverse-engineered.

• **Hand off the layers this build does not own.** SSO/SSL/keystore, Coherence/JMS/IWS tuning, batch

scheduling, and monitoring are separate solutions. Deliver a certified, healthy baseline install and let the specialized runbooks take it from there — keep this scope clean.

8. Deliverables Summary — Oracle Customer to Meter (C2M) Installation & Multi-Environment Build

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	The C2M/OUAF architecture (product home, WebLogic domain, database schema), the certified stack, the standard build model, benefits and limitations
2. Requirements Guide	Every C2M-specific prerequisite with Verify and Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Installation Guide	Exact ordered steps per environment: assess + backup, framework install, product install, database schema load, WebLogic domain, configure + validate, expand to Test then Prod
5. Testing Guide	Validation procedures with pass/fail gates, per environment
6. Troubleshooting Guide	Field issues at each build stage and safe remediation
7. Best Practices	Field-proven recommendations for repeatable multi-environment builds

Named objects (reused across the documents and scripts)

Object	Name	Scope
Environments (`$SPLENVIRON`)	`C2MDEV`, `C2MTST`, `C2MPRD`	Dev / Test / Prod
Product OS user	`cissys`	Owns $SPLEBASE, runs OUAF utilities
Product home / output	`$SPLEBASE` (`/opt/oracle/spl/your-env`) / `$SPLOUTPUT`	Per environment
WebLogic domain	`c2m_domain`	Native WebLogic 14c domain
WebLogic servers	`AdminServer`, `c2m_server1`	Admin + Managed server
DB accounts	`CISADM` (owner), `CISUSER` (app), `CISREAD` (read-only)	C2M schema
Security role	`cisusers`	Online application access

Automation scripts (WLST / shell / SQL) — five roles

Script	Language	What it does
`OC2MINST-Discovery.sh`	shell	Read-only discovery of the host/prereq state before a build: OS user, Java, WebLogic Home/version, mount points and free space, DB listener reachability, and whether the chosen WebLogic ports are free
`OC2MINST-Validation.py`	WLST/Jython	Read-only validation against `c2m_domain`: confirms `AdminServer` and `c2m_server1` are RUNNING and the C2M application/IWS are ACTIVE
`OC2MINST-Report.sql`	SQL	Read-only build/schema report: confirms `CISADM` owns the expected C2M objects, `CISUSER`/`CISREAD` exist, and the installed release matches the intended level
`OC2MINST-Remediation.sh`	shell	Safe remediation for common build issues (ownership/permissions on `$SPLEBASE`, template rebuild via `initialSetup.sh -t`, non-RUNNING server restart); **dry-run by default**, changes only with `-apply`

Script	Language	What it does
`OC2MINST-ScopeHelper.sh`	shell	Targeting/env helper: resolves `$SPLENVIRON`/`$SPLEBASE`/domain/DB for a named environment so commands run against the intended tier

The five roles are Discovery, Validation, Report, Remediation, and ScopeHelper, each authored in the language native to the layer it works on — shell for the OUAF file system and OS, WLST/Jython for the WebLogic domain, and SQL for the database schema.

What the buyer receives

- This installation and multi-environment build guide as a Word document and PDF.
- The automation scripts (WLST/Jython, shell, and SQL) in a single ZIP.
- A repeatable, per-environment process following the standard model: Assess → Configure → Pilot → Validate → Expand, with explicit pass/fail gates and a documented rollback.

Outcome

Oracle C2M installed and configured consistently in Dev, Test, and Production — certified stack, OUAF product home, healthy WebLogic `c2m_domain`, and a valid `CISADM` schema — validated against pass/fail gates and reproducible from recorded inputs, proven on non-production before Production is built.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MINST-Discovery.sh

Purpose: Captures the starting state of an application-tier host before a C2M install, read-only: OS product user, Java version, WebLogic Home and version, mount points and free space, database listener reachability, and whether the chosen WebLogic ports are free. Produces the Build Baseline record for one environment (your-env, e.g. C2MDEV). Changes nothing. HOW TO RUN Run as the product OS user (cissys) on the target host: ./OC2MINST-Discovery.sh -e your-env -w your-weblogic-home -d your-db-host -l 7001,7002,7003 -e environment name, -w WebLogic Home, -d DB listener host, -l comma-list of WebLogic ports. EXPECTED RESULTS A printed report of each prerequisite with an OK/CHECK marker; overall PASS if every item is OK, otherwise CHECK with the failing items listed. No files or systems are modified. NOTES White-label, repeatable pre-build recon. Read-only only. Oracle-version specifics (certified Java/WebLogic/DB versions) are compared informationally against values you pass in; confirm the certified combination against Oracle's certification matrix on My Oracle Support for your release.

How to run: Run as the product OS user (cissys) on the target host: ./OC2MINST-Discovery.sh -e your-env -w your-weblogic-home -d your-db-host -l 7001,7002,7003 -e environment name, -w WebLogic Home, -d DB listener host, -l comma-list of WebLogic ports. EXPECTED RESULTS A printed report of each prerequisite with an OK/CHECK marker; overall PASS if every item is OK, otherwise CHECK with the failing items listed. No files or systems are modified. NOTES White-label, repeatable pre-build recon. Read-only only. Oracle-version specifics (certified Java/WebLogic/DB versions) are compared informationally against values you pass in; confirm the certified combination against Oracle's certification matrix on My Oracle Support for your release.

OC2MINST-Remediation.sh

Purpose: Proposes and (only with -apply) performs safe remediation for common post-install problems on one environment (your-env, e.g. C2MDEV): incorrect ownership/permissions on the product home ($SPLEBASE), stale generated config templates (rebuild with initialSetup.sh -t), and a WebLogic server that is not RUNNING (offer a controlled restart via spl.sh). DRY-RUN by default: it prints the planned actions and changes nothing unless -apply is given. HOW TO RUN Preview (safe, default) as the product OS user (cissys): ./OC2MINST-Remediation.sh -e your-env Apply after reviewing the plan: ./OC2MINST-Remediation.sh -e your-env -apply Optional: -t fixperms | -t templates | -t restart to run one action only. EXPECTED RESULTS In dry-run: a numbered plan of the actions that WOULD run, ending with "DRY-RUN — nothing changed." With -apply: each action runs and reports OK/FAILED. Never touches Production without -apply, and never a server it is not asked to. NOTES White-label, repeatable remediation. Read-only until -apply. It calls documented OUAF utilities (splenviron.sh, initialSetup.sh, spl.sh) rather than editing generated files by hand; it never hand-edits ENVIRON.INI. Utility flags can vary by release — verify for your version against the OUAF Server Administration Guide.

How to run: Preview (safe, default) as the product OS user (cissys): ./OC2MINST-Remediation.sh -e your-env Apply after reviewing the plan: ./OC2MINST-Remediation.sh -e your-env -apply Optional: -t fixperms | -t templates | -t restart to run one action only. EXPECTED RESULTS In dry-run: a numbered plan of the actions that WOULD run, ending with "DRY-RUN — nothing changed." With -apply: each action runs and reports OK/FAILED. Never touches Production without -apply, and never a server it is not asked to. NOTES White-label, repeatable remediation. Read-only until -apply. It calls documented OUAF utilities

(splenviron.sh, initialSetup.sh, spl.sh) rather than editing generated files by hand; it never hand-edits ENVIRON.INI. Utility flags can vary by release — verify for your version against the OUAF Server Administration Guide.

OC2MINST-Report.sql

Purpose: Reports the state of a C2M application schema after the database install, read-only: confirms the schema owner (CISADM) exists and owns objects, that the application user (CISUSER) and read-only user (CISREAD) exist, counts the C2M schema objects, and reads the installed framework/product version rows. Run it as part of post-install validation and when comparing one environment (your-env) to another. It only SELECTs; it changes nothing. HOW TO RUN Connect as a read-only user that can see the data dictionary and the CISADM schema, then: sqlplus your-readonly-user/your-password@your-db @OC2MINST-Report.sql If your version stores installed-version data in different tables, adjust the marked section. EXPECTED RESULTS A printed report: account existence for CISADM/CISUSER/CISREAD, object counts by type owned by CISADM, invalid-object count (should be zero or explained), and the installed version rows. Pass when the accounts exist, CISADM owns the expected objects, and versions match your target. NOTES White-label, repeatable schema report. Read-only (SELECT only; no DDL/DML). Account names (CISADM/CISUSER/CISREAD) match the Installation Guide; change the DEFINE values below if you used different names. Version table/column names vary by release — the version section is marked "verify for your version" and is written to degrade gracefully if the table is absent.

How to run: Connect as a read-only user that can see the data dictionary and the CISADM schema, then: sqlplus your-readonly-user/your-password@your-db @OC2MINST-Report.sql If your version stores installed-version data in different tables, adjust the marked section. EXPECTED RESULTS A printed report: account existence for CISADM/CISUSER/CISREAD, object counts by type owned by CISADM, invalid-object count (should be zero or explained), and the installed version rows. Pass when the accounts exist, CISADM owns the expected objects, and versions match your target. NOTES White-label, repeatable schema report. Read-only (SELECT only; no DDL/DML). Account names (CISADM/CISUSER/CISREAD) match the Installation Guide; change the DEFINE values below if you used different names. Version table/column names vary by release — the version section is marked "verify for your version" and is written to degrade gracefully if the table is absent.

OC2MINST-ScopeHelper.sh

Purpose: Resolves the key coordinates of one C2M environment (your-env, e.g. C2MDEV, C2MTST, C2MPRD) so every command runs against the intended tier: the environment name ($SPLENVIRON), product home ($SPLEBASE), output dir ($SPLOUTPUT), WebLogic domain (c2m_domain), and the ENVIRON.INI-recorded database. It prints what it resolved and, for a Production target, requires explicit confirmation before it prints an "OK to proceed". Read-only: it inspects, it does not install or change. HOW TO RUN Print the resolved scope for an environment (as the product OS user cissys): ./OC2MINST-ScopeHelper.sh -e your-env Guard a Production run (must pass -confirm C2MPRD to get an OK-to-proceed): ./OC2MINST-ScopeHelper.sh -e C2MPRD -confirm C2MPRD EXPECTED RESULTS A printed scope block for the environment (env name, SPLEBASE, SPLOUTPUT, domain, DB) with each item marked OK or CHECK. Exit 0 and "OK to proceed" only when resolution succeeds and, for a Production target, the -confirm value matches. Otherwise a non-zero exit and no go-ahead. NOTES White-label, repeatable targeting guard — the way to avoid running a Dev/Test command against Production. Read-only. It reads ENVIRON.INI (generated by the installer) but never edits it. Directory conventions match the Installation Guide; adjust the bases below if you deviated.

How to run: Print the resolved scope for an environment (as the product OS user cissys): ./OC2MINST-ScopeHelper.sh -e your-env Guard a Production run (must pass -confirm C2MPRD to get an OK-to-proceed): ./OC2MINST-ScopeHelper.sh -e C2MPRD -confirm C2MPRD EXPECTED RESULTS A printed scope block for the environment (env name, SPLEBASE, SPLOUTPUT, domain, DB) with each item marked OK or CHECK. Exit 0 and "OK to proceed" only when resolution succeeds and, for a Production target, the -confirm value matches. Otherwise a non-zero exit and no go-ahead. NOTES White-label, repeatable targeting guard — the way to avoid running a Dev/Test command against Production. Read-only. It reads ENVIRON.INI (generated by the installer) but never edits it. Directory conventions match the Installation Guide; adjust the bases below if you deviated.

OC2MINST-Validation.py

Purpose: Connects to the C2M WebLogic 14c domain (c2m_domain) and confirms the build is healthy, read-only: the Administration Server (AdminServer) and Managed Server (c2m_server1) are RUNNING, and the C2M application plus inbound web services are ACTIVE. Makes no configuration change. Run it after the domain is created and the application deployed, and after any restart. HOW TO RUN Set the OUAF environment first (splenviron.sh -e your-env), then: java weblogic.WLST OC2MINST-Validation.py your-admin-url your-admin-user e.g. java weblogic.WLST OC2MINST-Validation.py t3://your-host:7001 weblogic The script prompts for the admin password (never pass it on the command line). EXPECTED RESULTS A printed check of each server and application state with OK/CHECK markers and an overall PASS/CHECK line. Exit 0 when every check passes, non-zero otherwise. Nothing is modified. NOTES White-label, repeatable domain validation. Read-only (connect + read runtime MBeans only). Server/application names (AdminServer, c2m_server1) match the Installation Guide; adjust them to your naming if you deviated. Exact MBean paths can vary by WebLogic version — verify for your version against Oracle's WebLogic 14.1.2 WLST documentation.

How to run: Set the OUAF environment first (splenviron.sh -e your-env), then: java weblogic.WLST OC2MINST-Validation.py your-admin-url your-admin-user e.g. java weblogic.WLST OC2MINST-Validation.py t3://your-host:7001 weblogic The script prompts for the admin password (never pass it on the command line). EXPECTED RESULTS A printed check of each server and application state with OK/CHECK markers and an overall PASS/CHECK line. Exit 0 when every check passes, non-zero otherwise. Nothing is modified. NOTES White-label, repeatable domain validation. Read-only (connect + read runtime MBeans only). Server/application names (AdminServer, c2m_server1) match the Installation Guide; adjust them to your naming if you deviated. Exact MBean paths can vary by WebLogic version — verify for your version against Oracle's WebLogic 14.1.2 WLST documentation.